

Engram: Memory Middleware for Multi-Agent AI Systems

Ananya Makwana
San Jose State University
ananya.makwana@sjsu.edu

Gayathri Rukmadhavan
San Jose State University
gayathri.rukmadhavan@sjsu.edu

Mahalakshmi Rajkumar
San Jose State University
mahalakshmi.rajkumar@sjsu.edu

Yukta Pramod Padgaonkar
San Jose State University
yuktapramod.padgaonkar@sjsu.edu

Abstract—Concurrent access to shared state in multi-agent artificial intelligence systems introduces silent data loss when last-write-wins semantics overwrite an agent’s decision without notification or recovery. Engram is a memory middleware service that addresses this problem through vector-clock conflict detection, deferred conflict handling via a Multi-Value Register CRDT, role-based access control, immutable history logging, and pluggable storage adapters. This paper presents a reconstruction of the Engram architecture, covering its conflict resolution strategies, rollback and snapshot query mechanisms, security assumptions, and testing approach. The resulting system enables autonomous agents to share mutable state while preserving causal ordering, exposing conflicts for deterministic or human-guided resolution, and maintaining a fully auditable write history.

Index Terms—multi-agent systems, memory middleware, vector clocks, CRDT, Multi-Value Register, RBAC, audit log, FastAPI, Redis

I. INTRODUCTION

Multi-agent AI systems increasingly coordinate specialized autonomous agents around shared plans, budgets, inventories, portfolios, or robot resources. In such settings, multiple agents may read and update the same logical memory key while operating with partial knowledge of other agents’ actions. Conventional key–value stores and in-memory dictionaries typically implement last-write-wins (LWW) semantics. Under LWW, the most recent write unconditionally overwrites the prior value, which is simple but unsafe for collaborative agent workloads.

The motivating failure mode is illustrated by a shared budget key. At time zero, Agent A writes `budget = $5000` with clock `{A:1}`. One millisecond later, Agent B, unaware of Agent A’s write, writes `budget = $3000` with clock `{B:1}`. In a LWW store, the resulting value is \$3000, and Agent A’s allocation is lost without an audit trail or conflict notification. In financial, inventory, and autonomous-control domains, this behavior creates correctness defects that are difficult to diagnose and expensive to remediate.

Engram addresses this problem as a middleware layer through which all shared-memory operations pass. Its objectives are to detect concurrent writes using vector clocks

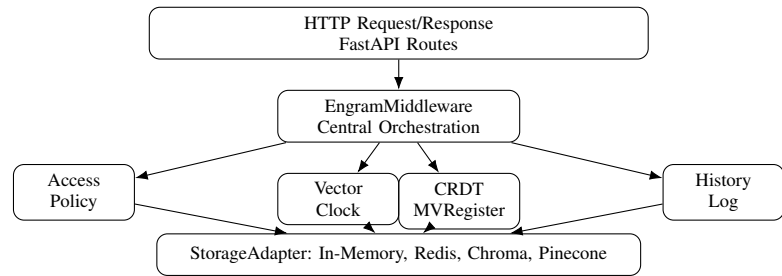


Fig. 1. Engram component interaction architecture.

[1], preserve concurrent values through a CRDT Multi-Value Register [2], defer resolution until a value is needed, enforce role-scoped access control, maintain immutable history, and support multiple storage backends including in-memory, Redis, and vector-database adapters.

II. SYSTEM ARCHITECTURE

A. Overview

Engram uses a middleware architecture centered on EngramMiddleware. The middleware composes access control, vector-clock management, CRDT state, history logging, and storage persistence. It is exposed through FastAPI endpoints and can run with an in-memory adapter during development or a Redis-backed adapter in production [3], [4].

B. Core Components

The middleware exposes three primary operations: `write(request)`, `read(key, agent_id, role)`, and `snapshot(key, at_clock)`. The write path first checks authorization, increments the relevant vector-clock component, retrieves the current register, performs causal comparison, logs history, and persists the updated register.

A vector clock is represented as a mapping from agent identifiers to counters. Incrementing a clock increases the writing agent’s counter; merging clocks takes the maximum counter per agent; comparison returns *before*, *after*, *equal*,

TABLE I
MVREGISTER WRITE SEMANTICS

Ordering	Register behavior
Empty	Add incoming value as the only candidate.
Before	Discard incoming value as stale.
After	Remove superseded values and store incoming value.
Concurrent	Preserve existing value and add incoming value.
Equal	Treat as duplicate or already-recorded state.

TABLE II
SUPPORTED RBAC KEY PATTERNS

Pattern	Meaning	Example match
*	All keys	transaction.history
budget.*	Prefix	budget.flights
transaction_*	Prefix	transaction_001
audit.log	Exact	audit.log
*.summary	Suffix	budget.summary

or *concurrent*. Two clocks such as {A:1} and {B:1} are concurrent because neither dominates the other.

Listing 1. Vector-clock comparison logic used by the Engram application.

```
class VectorClock:
    def increment(agent_id: str) -> VectorClock:
        clock[agent_id] = clock.get(agent_id, 0) + 1

    def merge(other: VectorClock) -> VectorClock:
        for agent in all_agents:
            clock[agent] = max(self[agent], other[agent])

    def compare(other: VectorClock) -> Ordering:
        # BEFORE, AFTER, CONCURRENT, or EQUAL
        ...
```

The Multi-Value Register (MVR) stores all causally concurrent values rather than selecting a winner. Each register entry contains the value, agent identifier, vector clock, and timestamp. If an incoming write is causally after an existing value, it supersedes that value. If it is before, it is stale and discarded. If it is concurrent, both values are retained and the key is marked conflicted.

C. Access Control and History

Engram uses role-based permissions with `fnmatch`-style wildcard patterns. Roles can be broad, such as `admin` with `*`, scoped, such as `budget-agent` with `budget.*`, or read-only, such as `auditor`. Permission checks are performed before reads and writes.

The history log is append-only and records an identifier, timestamp, key, value, agent identifier, vector clock, write type, and optional rollback target. Rollbacks create new rollback entries instead of mutating prior entries, preserving non-repudiation and enabling time-travel queries.

III. FUNCTIONAL CAPABILITIES

A. API Workflows

Engram provides endpoints for writes, reads, snapshots, history, rollbacks, role management, and health checks. The write endpoint accepts a key, value, agent identifier, role, vector

```
POST write
-> AccessPolicy.can_write(role, key)
-> VectorClock.increment(agent_id)
-> Storage.get(key)
-> MVRegister.write(value, clock)
-> HistoryLog.append(entry)
-> Storage.set(key, register)
-> return MemoryEntry
```

Fig. 2. Write operation flow in the Engram middleware pipeline.

TABLE III
CORE AND OPTIONAL TECHNOLOGIES

Technology	Version	Purpose	Role
FastAPI	0.111.0	Web framework	Async API
Uvicorn	0.29.0	ASGI server	Runtime
Pydantic	2.7.1	Validation	Models
Redis	5.0.4	Storage	Persistence
Python	3.11+	Language	Core implementation
ChromaDB	0.5.0	Vector DB	Semantic search
Pinecone	3.2.2	Managed vector DB	Scaling
pytest	8.2.0	Testing	Verification
pytest-asyncio	0.23.6	Async tests	Async support
ruff	Latest	Linting	Static checks

clock, and optional strategy indicator. A successful response returns the current value, previous value when applicable, updated clock, status, conflict flag, and timestamp. Permission failures return HTTP 403.

Reads check read permissions, return a single value when no conflict exists, and expose all values when a register is conflicted. Conflict resolution strategies include lowest value, highest value, latest clock, union, and flag-for-human. Snapshot queries support causal or timestamp-based inspection, e.g., retrieving the value of `budget.flights` when Agent A's counter was one. History queries return chronological writes and rollbacks. Role endpoints register, list, and update permissions. The health endpoint reports service status, storage backend, version, uptime, and operation count.

B. Example Multi-Agent Scenario

In the budget-management scenario, a flight agent and hotel agent share `budget.trip`. The flight agent writes \$5000 with clock {flight-agent:1}. The hotel agent concurrently writes \$3000 with clock {hotel-agent:1}. Since the clocks are incomparable, Engram stores both values, marks the key conflicted, and returns all values on subsequent reads. An administrator can then apply a strategy such as `highest_value`, producing \$5000 as the resolved value while preserving the full history.

IV. TECHNOLOGY STACK

Table III summarizes the implementation technologies used by the Engram application. FastAPI and Uvicorn provide the asynchronous HTTP layer; Pydantic provides request and response validation; Redis provides a production storage option; and Python 3.11+ supplies modern typing and asynchronous programming support [5], [6].

TABLE IV
TESTING STRATEGY AND REPORTED RESULTS

Layer	Example files	Share/Result
Unit	<code>test_vector_clock.py</code> , <code>test_crdt.py</code>	50%
Component	<code>test_storage.py</code> , <code>test_history.py</code>	40%
E2E	<code>test_e2e_budget_conflict.py</code>	10%
Coverage	All listed test groups	92% coverage

A. Architecture Decisions

The Engram design evaluates three major decisions. First, CRDT-based deferred resolution is selected over LWW and strict serialization because it preserves all concurrent values without imposing global coordination. Second, vector clocks are selected over wall-clock timestamps and sequence numbers because they represent causality without synchronized clocks. Third, role-based wildcard permissions are selected over no access control or full OAuth/JWT integration because Engram is designed as an internal middleware service with infrastructure-level authentication and lightweight per-operation authorization.

V. TESTING AND VALIDATION

Engram employs a layered test strategy with unit, component, and end-to-end tests. Unit tests cover vector-clock increment, merge, and comparison; CRDT single-write, concurrent-write, superseding, stale-write, and resolution cases; access-control wildcard and read-only behavior; and API success and failure paths. Component tests cover storage adapters, Redis serialization and TTL behavior, and history-log snapshots. The end-to-end budget conflict test verifies that concurrent writes preserve both values, conflict status is visible, resolution works, and history remains complete.

Representative test commands include `pytest tests/ -v`, targeted unit commands such as `pytest tests/test_vector_clock.py -v`, and coverage reporting through `pytest tests/ -cov=engram -cov-report=html` [7]. The Engram test suite validates vector-clock, CRDT, access-control, storage, history, and end-to-end behavior, with API tests covering primary success and failure paths and total coverage of 92%.

VI. SECURITY ANALYSIS

Engram assumes deployment in a trusted internal network protected by VPNs, private networks, TLS, mTLS, or an API gateway. It does not implement cryptographic client authentication internally. Instead, every request includes `agent_id` and `role`; authorization is enforced per operation; and audit trails record who attempted or performed state changes.

Primary vulnerabilities and mitigations include unauthorized writes, privilege escalation, history tampering, vector-clock spoofing, conflict-resolution injection, and replay attacks. Unauthorized writes are rejected by first-operation permission checks. Privilege escalation is mitigated by recommending an external gateway that binds authenticated identities to roles and overwrites client-supplied role fields. History tampering

TABLE V
DEPLOYMENT SECURITY CHECKLIST

Control	Recommendation
Network isolation	Deploy inside private VPC/security groups.
TLS/mTLS	Encrypt all client-to-Engram communications.
Gateway auth	Add JWT, OAuth2, or certificate validation upstream.
Rate limiting	Limit requests at the gateway.
Monitoring	Alert on 403 responses and unusual conflict rates.
Secrets	Store Redis passwords and API keys in a vault.
Audit export	Send history logs to a secure audit system.
CORS	Restrict origins if browser clients are used.

TABLE VI
REPORTED OPERATION LATENCIES

Operation	Latency	Notes
Write, no conflict	< 1 ms	In-memory operation
Write, conflict	1–5 ms	CRDT merge overhead
Read, single value	< 1 ms	Cache hit
Read, conflicted	1–3 ms	Vector-clock merge
Snapshot	5–50 ms	History scan
History query	10–100 ms	Log-size dependent

is mitigated by append-only logs and explicit rollback entries. Vector-clock spoofing is mitigated by server-maintained authoritative clocks rather than trusting inflated client clocks. Conflict-resolution injection is mitigated by server-side strategy configuration. Replay attacks are mitigated through vector-clock idempotency and duplicate or stale-write detection.

VII. PERFORMANCE CHARACTERISTICS

Engram is designed for low-latency middleware operation in internal multi-agent deployments. In-memory writes without conflicts are expected to complete in less than 1 ms because they require only authorization, vector-clock increment, register update, and local persistence. Conflicted writes add CRDT comparison and merge overhead and are expected to complete in the 1–5 ms range for typical small registers. Reads over a single value are cache-like lookups, whereas conflicted reads require merging clocks and applying a resolution strategy. Snapshot and history queries are more expensive because they scan the append-only history log; therefore, their latency grows with log size and storage-backend performance. These values should be interpreted as application-level design targets and observed operating ranges for development-scale deployments rather than hardware-independent guarantees. Production latency depends on Redis/network configuration, payload size, number of concurrent values per key, history retention policy, and API gateway overhead.

VIII. DISCUSSION AND FUTURE WORK

The project highlights several technical lessons. Vector clocks are powerful but require clear examples for adoption. Deferred conflict resolution preserves information that immediate winner selection would discard. Immutable history is

valuable for debugging and compliance. Wildcard permissions scale better than explicit ACL matrices for dynamic agent populations. Storage adapters make it practical to use in-memory stores during tests, Redis in production, and vector databases for semantic-memory extensions.

Near-term improvements include declarative YAML or JSON role configuration, Prometheus metrics, Grafana dashboards, and partial synchronization analysis. Medium-term work includes multi-instance Engram deployments with consensus, machine-learning-assisted semantic conflict resolution, anomaly detection, and a GraphQL query interface. Long-term extensions include cryptographic signing of history entries, WebSocket-based real-time synchronization, pub/sub conflict notifications, and multi-tenant isolation with quotas.

IX. CONCLUSION

Engram provides a practical middleware design for safe shared-memory operations in multi-agent AI systems. By combining vector clocks, a CRDT Multi-Value Register, RBAC, immutable history, and pluggable storage, it prevents silent data loss while preserving concurrency and auditability. The design is especially suitable for internal multi-agent deployments where infrastructure handles authentication and Engram provides causality-aware memory semantics. The Engram application demonstrates that the architecture supports conflict detection, data preservation, flexible resolution, access control, time-travel queries, and production-oriented deployment paths.

ACKNOWLEDGMENT

The authors thank Prof. Rakesh Ranjan for his guidance and feedback throughout this project. His mentorship was instrumental in shaping the system design and keeping the work on course. We also thank our peers for their suggestions, which improved the quality of the final implementation.

This work builds on the efforts of several open-source communities, including the FastAPI and Pydantic maintainers, the Redis project, the pytest ecosystem, and the distributed systems researchers whose foundational work on vector clocks and CRDTs made the conflict resolution model possible.

REFERENCES

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems*. Springer, 2011, pp. 386–400.
- [3] FastAPI Contributors, “Fastapi documentation,” <https://fastapi.tiangolo.com/>, 2025.
- [4] Redis Ltd., “Redis documentation,” <https://redis.io/docs/>, 2025.
- [5] Pydantic Contributors, “Pydantic documentation,” <https://docs.pydantic.dev/>, 2025.
- [6] Python Software Foundation, “asyncio — asynchronous i/o,” <https://docs.python.org/3/library/asyncio.html>, 2025.
- [7] pytest Contributors, “pytest documentation,” <https://docs.pytest.org/>, 2025.